Context Management

Table of Contents

- 1. Overview
  - 1.1 Architecture Layers Overview
  - 1.2 System Design Principles
- 2. Token Counting System
  - 2.1 Precise Counting vs. Estimation
  - 2.2 Rough Estimation (When Precise Data Unavailable)
  - 2.3 Hybrid Counting (Standard Function)
  - 2.4 Special Content Type Estimation
  - 2.5 Haiku-Assisted Counting
  - 2.6 Bedrock Special Handling
- 3. Context Window Management
  - 3.1 Core Constants
  - 3.2 Model Context Window Calculation
  - 3.3 Output Token Management
  - 3.4 Threshold Percentage System
  - 3.5 Blocking Limit
- 4. Microcompact
  - 4.1 Two Trigger Modes
  - 4.2 Microcompact Decision Flow
  - 4.3 Microcompact Warning Suppression
  - 4.4 API Layer Injection
- 5. Snip History Pruning
  - 5.1 How It Works
  - 5.2 Relationship with Compaction
  - 5.3 Snip Preservation Strategy
  - 5.4 Snip Trigger Conditions
- 6. API Layer Context Management
  - 6.1 Strategy Types
  - 6.2 Use Cases
- 7. Full Compaction
  - 7.1 Trigger Conditions
  - 7.2 Complete Execution Flow
  - 7.3 Compaction Summary Prompt Design
  - 7.4 Image Stripping
  - 7.5 Post-Compaction File Recovery
  - 7.6 Skill Recovery
  - 7.7 PTL Retry Mechanism
  - 7.8 Prefer Forking Sub-Agent for Compaction Summary

    - 7.9 Direct Streaming Path (Fallback)
- 8. Session Memory
    - 8.1 Core Difference from Compaction
    - 8.2 Trigger Conditions
    - 8.3 Sub-Agent Execution Flow
    - 8.4 Session Memory Template Explained
    - 8.5 Token Limits
    - 8.6 Custom Template Support
    - 8.7 Tool Usage Restrictions
    - 8.8 Mutual Exclusion with Main Agent
    - 8.9 Session Memory Usage
- 9. Session Memory Compaction
    - 9.1 Comparison with Traditional Compaction
    - 9.2 Preserved Message Calculation
    - 9.3 Feature Usage Determination
    - 9.4 Execution Flow
    - 9.5 Compaction Boundary Metadata
- 10. Auto-Compact
    - 10.1 Threshold System Explained
    - 10.2 Warning State
    - 10.3 Compaction Priority
    - 10.4 Circuit Breaker Explained
    - 10.5 Disable Control Matrix
    - 10.6 Marble Origami Special Case
- 11. Context Collapse
    - 11.1 Core Difference from Compaction
    - 11.2 Design Principles
    - 11.3 Mutual Exclusion with Auto-Compact
    - 11.4 Implementation Location
    - 11.5 Objects Being Collapsed
    - 11.6 Two-Phase Collapse Algorithm
    - 11.7 Relationship with Other Mechanisms
- 12. Context Analysis
    - 12.1 Use Cases
    - 12.2 Statistical Dimensions
    - 12.3 Duplicate Read Detection Algorithm
    - 12.4 Statsig Metric Mapping
    - 12.5 Detecting Optimizable Waste Points
    - 12.6 Summary
- 13. Post-Compaction Cleanup
    - 13.1 Cleanup Contents Explained
    - 13.2 Key Items NOT Cleared

- 13.3 Main Thread vs. Sub-Agent Distinction
- 14. Context Loading and Caching
  - 14.1 System Context
  - 14.2 Session Storage
- 15. Persistent Memory Extraction
  - 15.1 Difference from Session Memory
  - 15.2 Flow
  - 15.3 Storage Structure and Content
  - 15.4 Sub-Agent Execution
  - 15.5 Mutual Exclusion Protection
  - 15.6 Wait on Exit
- 16. History Management
  - 16.1 Key Constants
  - 16.2 Main Functions
- 17. Message Normalization and Boundary Management
  - 17.1 Message Normalization
  - 17.2 Compaction Boundary Messages
  - 17.3 Microcompact Boundary Messages
  - 17.4 Finding Compaction Boundaries
- 18. Key Design Decisions
  - 18.1 Why Fork a Sub-Agent for Summary?
  - 18.2 Why Microcompact at the API Layer Instead of Directly Modifying Messages?
  - 18.3 Why Use GrowthBook for Dynamic Configuration?
- 19. Global Data Flow Overview
  - 19.1 Complete Request Processing Flow
  - 19.2 Compaction Message Construction Flow
  - 19.3 Token Consumption by Context Layer

---

1. Overview

Claude Code's context management system is one of the most core and complex subsystems in the project. It consists of multiple layers of mechanisms, from low-level token counting, to mid-level microcompaction (tool result pruning), to high-level full compaction (conversation history summarization), and finally session-level and persistent-level memory management. The entire system revolves around one core goal: **maximize effective information within the limited context window**.

1.1 Architecture Layers Overview

```
┌──────────────────────────────────────────────────────────┐
│  Layer 5: Persistent Memory Extraction (extractMemories)│
│            → Cross-session persistence to ~/.claude/projects/│
├──────────────────────────────────────────────────────────┤
│  Layer 4: Session Memory                                 │
│            → Current session's Markdown notes            │
├──────────────────────────────────────────────────────────┤
```

```
│  Layer 3: Full Compaction                              │
│            → AI-generated history summary (calls model)│
├──────────────────────────────────────────────────────┤
│  Layer 2: History Pruning (Snip)                       │
│            → Selective discarding by time/Token       │
├──────────────────────────────────────────────────────┤
│  Layer 1: Microcompact                                 │
│            → Clear old tool results, keep prompt cache│
├──────────────────────────────────────────────────────┤
│  Layer 0: Token Counting & Threshold Detection         │
│            → Mathematical foundation for all decisions │
└──────────────────────────────────────────────────────┘
```

1.2 System Design Principles

- **Layered Management**: Each layer handles different granularity of problems (Token-level → Tool-level → Message-level → Session-level)
- **Conservative Triggers**: All automatic mechanisms have circuit breakers to prevent wasted API calls from repeated failures
- **Zero Information Loss Intent**: Compress history through structured summarization rather than simple discarding
- **Prompt Cache Friendly**: Compaction operations prioritize reusing existing API prompt caches
- **Progressive Optimization**: Prefer lightweight mechanisms first (MC → Snip → compaction), with heavyweight mechanisms as fallback

---

2. Token Counting System

Token counting is the foundation of the entire context management system; all compaction decisions are based on token counts.

2.1 Precise Counting vs. Estimation

Claude Code uses two token counting strategies, each suited for different scenarios:

Precise Counting (After API Call)

```ts
// utils/tokens.ts
getTokenUsage(message: Message): Usage | undefined

// Extracts from the usage field of assistant messages
// Conditions: message has usage field + not a synthetic message + not SYNTHETIC_MODEL
// Does not return progress/attachment/system type messages
```

The `Usage` object contains:

```ts
{
  input_tokens: number,
  output_tokens: number,
  cache_creation_input_tokens?: number,  // Cache creation cost
  cache_read_input_tokens?: number,       // Cache read savings
}
```

Precise Total Context Window Token Count

```
// utils/tokens.ts
getTokenCountFromUsage(usage: Usage): number
// = input_tokens + cache_creation_input_tokens + cache_read_input_tokens + output_tokens
// This number represents the full context size at the time of this API call
```

Getting Precise Count from Historical Messages

```
// utils/tokens.ts
tokenCountFromLastAPIResponse(messages: Message[]): number
// Traverses messages forward to find the first assistant message with usage
// Returns that message's getTokenCountFromUsage
// Used for token count comparison before/after compaction
```

Final Window Tokens for task_budget

```
// utils/tokens.ts
finalContextTokensFromLastResponse(messages: Message[]): number
// Gets final window size from usage.iterations[-1]
// (Used in server-side tool loop scenarios)
// Formula: input_tokens + output_tokens (excluding cache tokens)
```

2.2 Rough Estimation (When Precise Data Unavailable)

```
// services/tokenEstimation.ts
roughTokenCountEstimation(content: string, bytesPerToken = 4): number
// Simplest estimation: character count ÷ 4
// Suitable for plain text content
```

File-type-adjusted ratios:

```
// JSON/JSONL/JSONC files have byte/token ratio close to 2 (due to many single-character
symbols)
// Other files use default 4
bytesPerTokenForFileType(fileExtension: string): number {
  switch (fileExtension) {
    case 'json': case 'jsonl': case 'jsonc': return 2
    default: return 4
  }
}
```

2.3 Hybrid Counting (Standard Function)

```
// utils/tokens.ts
tokenCountWithEstimation(messages: readonly Message[]): number
```

This is the **most important counting function**, and is the basis for auto-compaction threshold decisions. Logic:

1. Traverse backward from the last message

2. Find the first assistant message with precise usage data
3. If found:
   a. First, traverse forward to include all split messages with the same message.id in the estimate (When model makes parallel tool calls, one API response produces multiple assistant messages)
   b. Precise usage + rough estimation of messages after
4. If not found: use rough estimation for all

> **Note**
>
> Calculate precisely what can be calculated precisely; for what cannot, perform rough estimation separately

Handling of Parallel Tool Calls: When the model makes multiple parallel tool calls in one response, the streaming code emits separate assistant message records for each content block (sharing `message.id` and `usage`), and immediately inserts tool_results between each tool_use. The message array structure is:

```
[..., assistant(id=A, usage), user(tool_result_A), assistant(id=A, usage),
 user(tool_result_B), assistant(id=A, usage), user(tool_result_C), ...]
```

If we only stop at the last assistant record, we miss all the intermediate tool_results that were inserted, leading to underestimation. The backtracking logic in `tokenCountWithEstimation` includes all records sharing the same `message.id` in the estimate.

> **Note**
>
> - **Blind spot in conventional counting logic:** Many statistical systems, when calculating token consumption for a conversation round or assembling historical messages, habitually **"look for the last assistant message"** as the endpoint of the current turn.
> - **Why underestimation occurs:** If the system only grabs that last `assistant(id=A, usage)` to settle the count, it ignores all the previous `assistant` records that also belong to `id=A`, and more importantly, misses the large text sandwiched in between — that is, the `tool_result`s (tool results typically contain a large number of tokens). This causes the system to calculate far fewer tokens than actually consumed (underestimation).

2.4 Special Content Type Estimation

```ts
// services/tokenEstimation.ts
roughTokenCountEstimationForBlock(block): number

// Text blocks: character count ÷ 4
if (block.type === 'text')
  return roughTokenCountEstimation(block.text)

// Image/document blocks: fixed 2000 tokens
// Reason: Claude platform charges for images as (width × height) / 750
// Cap: 2000×2000 = 5333 tokens, conservative value 2000
if (block.type === 'image' || block.type === 'document')
  return 2000  // IMAGE_MAX_TOKEN_SIZE

// tool_use: tool name + JSON-serialized input parameters
```

```
// Note: input here may be arbitrarily large (bash commands, file contents, etc.)
// Directly stringify once for estimation, because API will re-serialize anyway
if (block.type === 'tool_use')
  return roughTokenCountEstimation(
    block.name + JSON.stringify(block.input ?? {})
  )

// tool_result: recursively compute its content
if (block.type === 'tool_result')
  return roughTokenCountEstimationForContent(block.content)

// thinking blocks: only count thinking text, excluding JSON wrapper and signature
if (block.type === 'thinking')
  return roughTokenCountEstimation(block.thinking)

// Redacted thinking: compute data field
if (block.type === 'redacted_thinking')
  return roughTokenCountEstimation(block.data)

// All other block types (server_tool_use, web_search_tool_result, etc.)
// Unified stringify + estimation
return roughTokenCountEstimation(JSON.stringify(block))
```

2.5 Haiku-Assisted Counting

```
// services/tokenEstimation.ts
countTokensViaHaikuFallback(messages, tools): Promise<number>
```

When precise usage is unavailable (e.g., after cross-session recovery), Claude Code uses Haiku (the fastest model) to estimate token count:

```
const model = isVertexGlobalEndpoint || isBedrockWithThinking || isVertexWithThinking
  ? getDefaultSonnetModel()   // Vertex global endpoint or Bedrock+thinking: use Sonnet
  : getSmallFastModel()         // Otherwise use Haiku 4.5 (supports thinking)
// Calls anthropic.beta.messages.create (max_tokens=1 or 2048)
// Returns input_tokens + cache_creation + cache_read
```

2.6 Bedrock Special Handling

```
// services/tokenEstimation.ts
countTokensWithBedrock({ model, messages, tools, betas, containsThinking })

// Bedrock's CountTokens API requires base model ID (not inference profile ARN)
// Needs conversion via getInferenceProfileBackingModel()
// Model: bedrock-2023-05-31
```

---

3. Context Window Management

3.1 Core Constants (`utils/context.ts`)

```
MODEL_CONTEXT_WINDOW_DEFAULT = 200_000    // Default context window: 200K tokens
COMPACT_MAX_OUTPUT_TOKENS = 20_000        // Max output for compaction model calls
ESCALATED_MAX_TOKENS = 64_000            // Max output in escalated mode
CAPPED_DEFAULT_MAX_TOKENS = 8_000        // Default capped mode max output
```

3.2 Model Context Window Calculation

```
getContextWindowForModel(model: string, betas?: string[]): number
// Standard models: 200K
// 1M-supported models (via has1mContext / modelSupports1M): 1,000,000
// Special beta config: may be smaller

getEffectiveContextWindowSize(model: string): number
// = getContextWindowForModel(model) - MAX_OUTPUT_TOKENS_FOR_SUMMARY(20K)
// Reserve 20K for model output; actual usable input window
// Supports env var override: CLAUDE_CODE_AUTO_COMPACT_WINDOW=150000
```

3.3 Output Token Management

```
getModelMaxOutputTokens(model: string): number
// Different models have different output limits:
// Sonnet 4.6: 64K
// Sonnet 4.5: 32K
// Haiku: 8K
// ...
```

3.4 Threshold Percentage System

```
Effective Context Window (effectiveWindow)
   ┌──────────────────────────────────────────┬────────────────┐
   │                                          │                │ ← Output Buffer (20K)
   │  0%              Warning Line    93.5%│  100%   100% │
   │     ├──────────────┤├────────────────┤   │                │
   │     WARNING_BUFFER  AUTO_BUFFER         │ Overflow!      │
   │      (20K)         (13K)               │                │
   └──────────────────────────────────────────┴────────────────┘

           ↑
     calculateTokenWarningState(tokenUsage, model)
     Returns:
       percentLeft        = remaining percentage
       isAboveWarningThreshold      // Warning
       isAboveErrorThreshold        // Error
       isAboveAutoCompactThreshold  // Auto-compact
       isAtBlockingLimit            // Blocking
```

3.5 Blocking Limit

```
// Beyond this threshold, block user from continuing input (prevent overflow)
isAtBlockingLimit = tokenUsage >= blockingLimit
// blockingLimit = effectiveWindow - MANUAL_COMPACT_BUFFER_TOKENS(3K)
// Approximately effectiveWindow - 3K
```

```
// Supports env var override: CLAUDE_CODE_BLOCKING_LIMIT_OVERRIDE
```

---

4. Microcompact

Microcompact is the lightest-weight context management mechanism executed before each API call. It does not summarize but directly clears or replaces old tool result content.

> **Note**
>
> Actually does tool result pruning

4.1 Two Trigger Modes

4.1.1 Time-Based Microcompact

Core idea: Claude API's Prompt Cache has a 1-hour (60-minute) TTL. When more than 60 minutes have passed since the last Assistant message, **the cache has expired** and the entire prefix will be recalculated. At this point, **preemptively clearing old tool results can reduce the data volume that needs to be rewritten**.

> **Note**
>
> - Whether cache hits depends not only on whether the prefix is the same, but also on TTL status
> - **TTL is timed by measuring the time difference between two requests** (when a new request arrives, the timer resets)

```
// services/compact/timeBasedMCConfig.ts
const TIME_BASED_MC_CONFIG_DEFAULTS = {
  enabled: false,               // Disabled by default, enabled via GrowthBook
'tengu_slate_heron'
  gapThresholdMinutes: 60,    // 60 minutes (aligned with server cache TTL)
  keepRecent: 5,               // Keep the 5 most recent compactable tool results
}
```

Trigger Evaluation:

```
evaluateTimeBasedTrigger(messages, querySource): { gapMinutes, config } | null
// Must satisfy:
// 1. config.enabled = true (GrowthBook switch)
// 2. querySource === 'repl_main_thread' (main thread only)
// 3. Has a most recent assistant message
// 4. gap = (now - lastAssistant.timestamp) / 60000 >= config.gapThresholdMinutes
```

Processing Logic (`maybeTimeBasedMicrocompact`):

1. `collectCompactableToolIds()` collects all compactable tools' tool_use_ids (in order of appearance) -> also sets which tools' results are compactable; by default, key tools like AgentTool and TaskOutputTool are NOT in this list and won't be cleared
2. Keep the most recent N (keepRecent), add the rest to `clearSet` -> Pruning protection

3. Iterate through messages, replace `clearSet` tools' tool_result content with `[Old tool result content cleared]`
4. **Key point**: Modifying prompt content breaks server-side cache, so reset Cached MC state and notify cache break detection -> Cache already expired, no KV cache miss cost increase

Cleanup Scope (`COMPACTABLE_TOOLS`):

```
const COMPACTABLE_TOOLS = new Set([
  'Read',
  'Bash', 'PowerShell',  // ...all SHELL_TOOL_NAMES
  'Grep',
  'Glob',
  'WebSearch',
  'WebFetch',
  'Edit',
  'Write',
])
```

4.1.2 Cached Microcompact

Core idea: Leverage the API's `cache_edits` mechanism by specifying `cache_reference` (referencing existing cached segments) and `cache_edits` (edit operations) in the request to delete **old tool results** while **preserving the validity of the prompt cache**.

> **Note**
>
> - Advantage: Using cache_edit for deletion effectively masks part of the KV, improving context quality without breaking KV cache
> - These are fields considered in session modeling; they don't appear in context (all infra-layer instruction parameters are in session, not in context)

```
┌──────────────────────────────────────────────────────────────────┐
│  Server-side Cache (Prompt Cache)                                 │
│                                                                   │
│  [System Prompt + Tool Definitions + Message Prefix]  ← Cached, cache_hit │
│                                                                   │
│  [Tool Result A: Read foo.py]           ← cache_edit: delete      │
│  [Tool Result B: Bash git status]       ← cache_edit: delete      │
│  [Tool Result C: Grep "func"]           ← kept              │
│  [Latest User Message]                                              │
└──────────────────────────────────────────────────────────────────┘
                              ↓
                     API request only transmits delta
                     (which to delete, not everything)
```

Enable Conditions (`microCompact.ts` 's `cachedMicrocompactPath`):

1. Feature flag `CACHED_MICROCOMPACT` is on
2. Model supports cache editing (`isModelSupportedForCacheEditing(model)`) -> checks if model service supports this feature (decided by infra layer)

3. `isMainThreadSource(querySource)` is true (`querySource === 'repl_main_thread'` or starts with this prefix)

State Tracking (`cachedMicrocompact.js` , dynamic import):

```
CachedMCState = {
  toolOrder: string[],           // All registered tool_use_ids (in order)
  registeredTools: Set<string>,  // Registered tool_use_ids
  deletedRefs: Set<string>,      // Deleted tool_use_ids
  pinnedEdits: PinnedCacheEdits[],  // Pinned edit operations
}
```

Workflow:

1. **Register**: In each user message, for each new tool_result, call `registerToolResult(tool_use_id)` , grouped by user message
2. **Evaluate**: Call `getToolResultsToDelete()` , based on GrowthBook-configured thresholds to determine which to delete
3. **Pin**: If edit detection finds an edit (tool_result of an edit tool), pin the edit operation (`pinCacheEdits`), ensuring it's sent on the next request
4. **Execute**: Create `cache_edits` block, marking tool_use_ids to delete
5. **API Layer Injection**: After `microCompactMessages` returns, API layer adds `cache_reference` and `cache_edits`

4.2 Microcompact Decision Flow

```
// services/compact/microCompact.ts
microcompactMessages(messages, toolUseContext?, querySource?): MicrocompactResult

// Execution order (short-circuit logic):
async function microcompactMessages(messages, toolUseContext, querySource) {
  clearCompactWarningSuppression()  // Clear warning suppression

  // 1. Time-based MC takes priority (cache expired detected)
  const timeResult = maybeTimeBasedMicrocompact(messages, querySource)
  if (timeResult) return timeResult  // Short-circuit: time-based first

  // 2. Cache edit MC (conditions must be met)
  if (feature('CACHED_MICROCOMPACT') && isCachedMCEnabled() && isModelSupported() &&
isMainThread) {
    return cachedMicrocompactPath(messages, querySource)  // Short-circuit
  }

  // 3. No available MC path → return original messages (fallback)
  return { messages }
}
```

4.3 Microcompact Warning Suppression

```
// services/compact/compactWarningState.ts
compactWarningStore: Store<boolean>
```

```
// Set suppression=true immediately after successful microcompact
// UI layer subscribes to this state via useCompactWarningSuppression() hook
// When suppression=true, don't show "context about to compact" warning
// Reason: After microcompact, the client doesn't know exactly how large the context is now;
// the next API response will have precise data, so showing "context about to compact"
warning
// with an uncertain specific value is avoided
```

4.4 API Layer Injection

When constructing the API request (`microCompact.ts` → caller), pending `cache_edits` are retrieved via `consumePendingCacheEdits()` and injected into the request. At the same time, `cache_deleted_input_tokens` is extracted from the last assistant message as a baseline value for computing the delta.

---

5. Snip History Pruning (Similar to context truncation, disabled by default)

Snip (`HISTORY_SNIP` feature flag) is a mechanism that directly discards early content from the message history without summarizing. Unlike compaction, Snip discards content the model no longer needs to see (e.g., file read results that have already been processed).

5.1 How It Works

```
Message History
  [Msg0] [Msg1] ... [MsgN-2] [MsgN-1] [Latest Msg]
      ↑                               ↑
   snip point ← ← ← ← ← ← ← ← ← ← ← ← keep tail

After Snip:
  [SNIP_MARKER] [MsgN-2] [MsgN-1] [Latest Msg]
```

5.2 Relationship with Compaction

- **Can coexist**: Snip and compaction are not mutually exclusive and can trigger simultaneously
- **Execution order**: Snip executes before microcompact (`query.ts:396`)
- **Token Release Tracking**: `snipTokensFreed` is passed to `autoCompactIfNeeded`, ensuring auto-compaction threshold judgment reflects the space Snip has freed

```
// In query.ts:
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage
  }
}
// ... subsequent microcompact and autocompact use snipTokensFreed
```

5.3 Snip Preservation Strategy

Snip does not discard indiscriminately; it preserves:

1. **Most recent N messages**: Ensures working context is not disrupted
2. **Messages containing tool_uses without paired tool_results**: Prevents orphan tool_results
3. **Session boundary messages**: Such as compaction boundaries, SessionStart messages, etc.

> **Note**
>
> Directly modifies messages[], replacing an intermediate segment with a placeholder marker, then only sends the later portion to the API -> Equivalent to truncating the head
> Is a compiled dynamic module loaded via require(), so the specific trigger timing is unknown; from code structure it appears to be count-based (alternatively, can be manually triggered by user via `/force-snip` command)

5.4 Snip Trigger Conditions

1. Feature Flag switch

```
// Conditional compilation switch — HISTORY_SNIP enabled only when true
feature('HISTORY_SNIP')
```

2. Trigger Method 1: Automatic check every query loop (auto-trigger)

```
User Input
    ↓
query.ts:401 —必经之路 every API call before
    ↓
snipModule!.snipCompactIfNeeded(messagesForQuery)
    ↓
[Condition Check] Need snip?
    ↓
May produce: messages + tokensFreed + boundaryMessage
```

Core call site query.ts:401-410:

```
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages   // Pruned messages
  snipTokensFreed = snipResult.tokensFreed  // Freed token count
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage        // Emit boundary marker
  }
}
```

Execution order: Executes before microcompact (query.ts:396 comment)

3. Trigger Method 2: Model actively calls snip tool (tool-call triggered)

SNIP is exposed to the model as a tool (SnipTool):

```
// tools.ts:123
const SnipTool = feature('HISTORY_SNIP')
  ? require('./tools/SnipTool/SnipTool.js').SnipTool
  : null
```

The model can proactively call the snip tool in any turn, telling Claude Code which history segment to prune. Tool call triggers `snipCompactIfNeeded({ force: true })` (see QueryEngine.ts:1238 snipReplay logic).

4. Trigger Method 3: Manual command /snip

```
// commands.ts:83
const forceSnip = feature('HISTORY_SNIP')
  ? require('./commands/force-snip.js').default
  : null
```

When HISTORY_SNIP is on, users can manually invoke the /snip command to force-trigger pruning.

5. Nudge Mechanism: Periodic reminders for the model to use snip

```
// attachments.ts:3963
export function getContextEfficiencyAttachment(messages: Message[]): Attachment[] {
  if (!isSnipRuntimeEnabled()) return []
  if (!shouldNudgeForSnips(messages)) return []  // ← Trigger check
  return [{ type: 'context_efficiency' }]         // ← Inject reminder
}
```

---

6. API Layer Context Management

When the model API **supports native context management strategies**, you can **declare context management strategy in the request's context_management field**, letting the server's inference engine automatically perform cleanup.

> **Note**
>
> Essentially **partially delegates context management decision-making to the server**, letting the server handle it at the right time automatically, without the client repeatedly sending cache_edits or doing content replacement

6.1 Strategy Types (`services/compact/apiMicrocompact.ts`)

```
type ContextManagementConfig = {
  edits: ContextEditStrategy[]
}
```

6.1.1 `clear_tool_uses_20250919`

```
{
  type: 'clear_tool_uses_20250919',
  trigger: {
    type: 'input_tokens',
```

```
    value: 180_000  // Trigger threshold (DEFAULT_MAX_INPUT_TOKENS)
  },
  keep: {
    type: 'tool_uses',
    value: N         // Keep the most recent N tool calls
  },
  clear_tool_inputs: [
    'Bash', 'Read', 'Grep', 'Glob', 'WebSearch', 'WebFetch'
  ],
  clear_at_least: {
    type: 'input_tokens',
    value: 140_000  // Clear at least 140K tokens (180K - 40K)
  },
  exclude_tools: ['Edit', 'Write', 'NotebookEdit']  // Don't clear edit-type tools
}
```

Trigger timing: When `input_tokens >= trigger.value`, the server automatically executes cleanup down to the `keep` limit.

6.1.2 `clear_thinking_20251015`

```
{
  type: 'clear_thinking_20251015',
  keep: 'all' | { type: 'thinking_turns', value: N }
// Keep the most recent N thinking blocks (or all)
// Skip when redact-thinking is active (because redacted blocks have no model-visible
content)
}
```

6.2 Use Cases

- **Thinking cleanup**: Used by default in all sessions with thinking enabled but redact-thinking not enabled
- **Tool result cleanup**: Enabled via `USE_API_CLEAR_TOOL_RESULTS` / `USE_API_CLEAR_TOOL_USES` environment variables (ANT users only)

> **Note**
>
> - redact-thinking: "Redact" thinking blocks — replace model-generated thinking content with unreadable redacted_thinking blocks. Mainly for privacy/security and token billing issues
> - ANT users: Abbreviation for Anthropic internal employees, can test experimental features

---

7. Full Compaction

When all lightweight mechanisms are insufficient to control context size, Claude Code executes full compaction: generating a structured summary using an AI model, replacing the entire conversation history.

7.1 Trigger Conditions

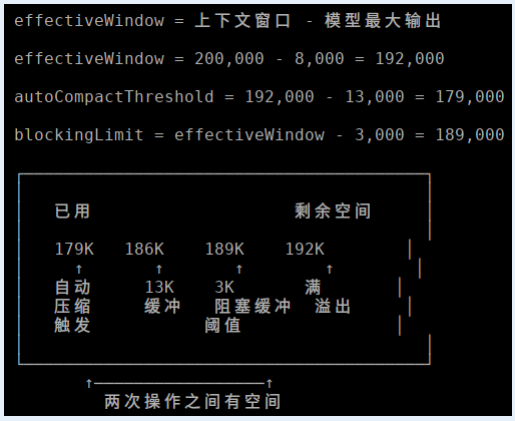
Triggered by token count

```
tokenCountWithEstimation(messages) >= getAutoCompactThreshold(model)
// = effectiveContextWindow - AUTOCOMPACT_BUFFER_TOKENS(13K)
```

```
// Approximately 93.5% of effectiveWindow
```

Note

effectiveWindow = model context window - model max output tokens



7.2 Complete Execution Flow

```
Input: messages[] (current complete conversation history)
Output: CompactionResult

Phase 1: Pre-processing
  ↓
  executePreCompactHooks()  ← Execute pre-compaction hooks (allow custom directives)
  ↓
Phase 2: Generate Summary (call AI model)
  ↓
  stripImagesFromMessages()   ← Strip images/documents
  stripReinjectedAttachments()← Strip reinjectable attachments (two types: skill_discovery
                                skill discovery signal (tells model which skills are
available)
                                and skill_listing content. Stripped because these
attachments
                                will be reinjected after compaction)
  ↓
  streamCompactSummary()       ← Call model to generate summary
      ├── Priority: fork sub-agent path (shared prompt cache)
      └── Fallback: direct streaming call (streaming fallback)
  ↓
  PTL retry (up to 3 times):
      truncateHeadForPTLRetry()  ← If the compaction request itself exceeds context
  ↓
Phase 3: State Cleanup
  ↓
  context.readFileState.clear()         ← Clear file read cache (after each Read tool call,
                                          file content is cached. Clearing is for all files
                                          not selectively, done to: 1. prevent duplication
                                          between compaction result and re-read content;
                                          2. prevent file content already modified by user
                                          or environment; force reading latest version;
                                          3. after compaction, conversation history becomes
```

```
                                          summary, model no longer needs the content of
files
                                          it "claims to have read")

  context.loadedNestedMemoryPaths.clear()← Clear nested memory paths (this Set records which
                                          nested CLAUDE.md files have been injected into the
                                          conversation during this session; clearing is for
                                          deduplication)
  ↓
Phase 4: Re-inject Attachments (post-compaction info backfill)
  ↓
  Recent state, maintain semantic continuity
  createPostCompactFileAttachments()    ← Restore recently accessed files (max 5, 50K total)

  Restore state
  createAsyncAgentAttachmentsIfNeeded()  ← Restore async agent state
  createPlanAttachmentIfNeeded()         ← Restore plan file
  createPlanModeAttachmentIfNeeded()   ← Restore plan mode reminder
  createSkillAttachmentIfNeeded()       ← Restore invoked skills (5K per skill), with skill
                                          invocation state

  Restore environment info
  getDeferredToolsDeltaAttachment()      ← Re-announce tool changes
  getAgentListingDeltaAttachment()        ← Re-announce agent list
  getMcpInstructionsDeltaAttachment()  ← Re-announce MCP instructions
  ↓
Phase 5: Create Compaction Boundary
  ↓
  createCompactBoundaryMessage()        ← Create system boundary message (update location
                                          marker, marking which content before it was
                                          compacted)
  ↓
Phase 6: SessionStart Hooks
  ↓
  processSessionStartHooks('compact')  ← Reload CLAUDE.md and other context (this alone
                                          after creating compaction boundary is because
                                          modeling is different: tool/agent/MCP delta are
                                          built in real-time during compaction, injected
into
                                          the messages array; Memory / CLAUDE.md are frozen
                                          in systemPrompt before compactConversation() is
                                          called. If changed during compaction, it only
takes
                                          effect on the next API request — so just wait for
                                          the next round. It's not for the current API call,
                                          but for the next getSystemPrompt() call — ensuring
                                          the next round gets fresh memory content)
  ↓
Phase 7: Event Logging and Cleanup
  ↓
  notifyCompaction()                    ← Notify cache break detection (to prevent cache
break
                                          detection from false positives after compaction:
```

```
                                          after compaction, message count legitimately drops
                                          significantly, cache_read_tokens will drop
                                          significantly — this is normal behavior, not a
break)
  markPostCompaction()                  ← Set post-compaction state
  reAppendSessionMetadata()             ← Refresh session metadata
  writeSessionTranscriptSegment()       ← Write session record (KAIROS feature)
  executePostCompactHooks()             ← Execute post-compaction hooks
  runPostCompactCleanup()               ← Cleanup cache and state
```

7.3 Compaction Summary Prompt Design

The compaction summary requires the model to generate highly structured output; this is one of the most critical parts of the entire compaction system.

7.3.1 Prompt Template Structure

The prompt has three parts:

1. Mandatory No-Tools Lead (CRITICAL)

```
CRITICAL: Respond with TEXT ONLY. Do NOT call any tools.
- Do NOT use Read, Bash, Grep, Glob, Edit, Write, or ANY other tool.
- You already have all the context you need in the conversation above.
- Tool calls will be REJECTED and will waste your only turn — you will fail the task.
- Your entire response must be plain text: an <analysis> block followed by a <summary>
block.
```

Extremely Important: Only reply with plain text. Do not call any tools under any circumstances.

- Do not use Read, Bash, Grep, Glob, Edit, Write, or any other tools.
- You already have all the context you need in the conversation above.
- Tool calls will be rejected and will waste your only turn — this will cause your task to fail.
- Your entire response must be plain text: one block followed by another block.

Design rationale: The compaction fork sub-agent inherits the parent session's complete toolset, but the model may still attempt tool calls. This lead explicitly prohibits tool use, ensuring the model only outputs text.

> **Note**
>
> When testing OpenClaw's compaction prompt, it was found that the model had some probability of not following compaction instructions and instead choosing to continue executing the next task. Only when the compaction agent system prompt and query structure were changed did compaction instruction following improve. CC doesn't have this problem, possibly due to model intelligence differences.

2. Detailed Analysis Instructions (DETAILED_ANALYSIS_INSTRUCTION)

The model is required to do the following in the `<analysis>` block:

- Analyze each message in chronological order -> Overcome model's "attention loss" and "context forgetting" issues in long conversations, preventing logical discontinuities
- Identify: explicit requests, intent, code snippets, filenames, function signatures -> Entity extraction, convert to structured information

- Record: encountered errors, user corrections, methods no longer tried -> Negative memory, prevent repeated pitfalls
- Ensure technical accuracy and completeness

3. Summary Format Requirements (9 Required Sections)

> **Note**
>
> OpenClaw, as a more general-purpose agent, has similar other sections (mainly user request info, environment info, task status info), but **important code snippets** etc. are replaced with **important context snippets**

```
1. Primary Request and Intent:
   All explicit requests and intents from the user

2. Key Technical Concepts:
   Important technical concepts, frameworks, pattern list

3. Files and Code Sections:
   - Filename
     - Why this file is important
     - What modifications were made to this file (if any)
     - Important code snippets
   - ...

4. Errors and Fixes:
   - Error description:
     - Fix method
     - User feedback on this error (if any)
   - ...

5. Problem Solving:
   Problems already solved + ongoing investigation

6. All User Messages:
   All non-tool-result user messages (chronological order)

7. Pending Tasks:
   Explicitly requested but not yet completed tasks

8. Current Work:
   Specific work in progress before compaction
   (Includes filenames, code snippets)

9. Optional Next Step:
   Next step that directly continues recent work
   (Must reference specific content from the original conversation)
```

7.3.2 Output Parsing

```
// Extract summary from model response in compact.ts:
const summary = getAssistantMessageText(summaryResponse)

// <analysis> block is stripped during parsing, only <summary> portion kept
// Because <analysis> is a draft process and should not enter context
```

7.3.3 Summary Result Storage

```
// Summary stored as user message (role='user')
createUserMessage({
  content: getCompactUserSummaryMessage(summary, suppressQuestions, transcriptPath),
  isCompactSummary: true,              // Marked as compaction summary
  isVisibleInTranscriptOnly: true,   // Only visible in transcript, not sent to API
  summarizeMetadata: {
    messagesSummarized: N,
    userContext: string,
    direction: 'from' | 'up_to',
  }
})
```

Note: `isVisibleInTranscriptOnly` ensures the summary **does not enter the next API request** (normalizeMessagesForAPI filters it out), but appears in the user-visible transcript.

7.4 Image Stripping

```
stripImagesFromMessages(messages: Message[]): Message[]
```

Images in user messages are replaced before compaction: (**placeholder replacement**)

- `image` blocks → `[image]` text block
- `document` blocks → `[document]` text block
- **Images/documents nested in tool results**: handled the same way

Rationale: Images don't directly help generate conversation summaries. But images can cause the compaction API call itself to exceed context limits (in CCD sessions users frequently attach images), so text markers are used as substitutes.

> **Note**
>
> stripImagesFromMessages only takes effect in the streaming fallback, and the streaming fallback doesn't use the main conversation's KV cache at all, so it has no impact on main conversation KV cache hit rate

7.5 Post-Compaction File Recovery

```
createPostCompactFileAttachments(
  preCompactReadFileState,  // All file reads tracked before compaction
  context,
  maxFiles: 5,              // POST_COMPACT_MAX_FILES_TO_RESTORE
  preservedMessages?,       // Preserved messages (avoid duplicate injection)
): AttachmentMessage[]
```

File Selection Strategy:

1. Extract all files from `readFileState` (sorted by most recent access time)
2. Exclude plan files (`getPlanFilePath()`) and all memory files
3. Exclude files that already have Read results in `preservedMessages` (avoid duplicates)
4. Take the most recent 5

Token Budget Control:

```
POST_COMPACT_TOKEN_BUDGET = 50_000   // Max 50K tokens total
POST_COMPACT_MAX_TOKENS_PER_FILE = 5_000  // Max 5K per file
// When file exceeds 5K, truncate from the beginning, preserving a marker
```

Token Savings from File Recovery: Without recovery, after each compaction the model would need to re-read these files. At an average of 500 tokens per Read and 5 files, each compaction saves approximately 2.5K tokens.

7.6 Skill Recovery

```
createSkillAttachmentIfNeeded(agentId?): AttachmentMessage | null

// Skill content is key information that persists across compactions
// Max 5K tokens per skill
// Total skill budget 25K tokens
// Sorted by invocation time (newest first), discard oldest when exceeding budget
```

Why not clean sentSkillNames: Re-injecting the complete skill listing (~4K tokens) is pure cache creation with no actual benefit. Invoked skill content is preserved through `invoked_skills` attachment, and new skill dynamic additions are handled by `skillChangeDetector`.

> **Note**
>
> - InvokedSkillInfo retains "invocation timestamp" and "which agent called it"; invoked skills retain the original version because after compaction, the skill file path remains the same, but the file content may have been modified by the user; directly storing a snapshot at compaction time ensures consistency
> - InvokedSkill tracks skills that have already been called; a skill is recorded the moment it's invoked
> - invokedSkills stores the static content of skills (file text + argument substitution); argument substitution refers to the $ARGUMENTS placeholder being replaced with actual parameters passed when the skill was called

7.7 PTL Retry Mechanism

> **Note**
>
> - Uses 135K as a fixed value to trigger prompt-too-long judgment
> - `tokenGap` refers to when the compaction API request itself triggers PTL (Prompt Too Long), the API error message carries how many tokens over

```
// When the compaction request itself triggers prompt_too_long
truncateHeadForPTLRetry(messages, ptlResponse): Message[] | null

// Max 3 retries (MAX_PTL_RETRIES = 3)
// Retry strategy:
  // 1. Parse tokenGap from API error
  // 2. Group by API round, discard from the oldest group first (one conversation round =
one group)
  // 3. Discard until tokenGap is covered
  // 4. Fallback: if tokenGap cannot be parsed, discard 20% of groups directly

// Retry flag: '[earlier conversation truncated for compaction retry]'
// If the first message is this flag, strip it before grouping (avoid infinite loop)
```

7.8 Prefer Forking Sub-Agent for Compaction Summary

```
// compact.ts - streamCompactSummary priority path
if (promptCacheSharingEnabled) {
  const result = await runForkedAgent({
    promptMessages: [summaryRequest],
    cacheSafeParams,        // Inherit parent session's cache params
    canUseTool: createCompactCanUseTool(),  // Disable tools
    querySource: 'compact',
    forkLabel: 'compact',
    maxTurns: 1,            // Only allow 1 turn
    skipCacheWrite: true,   // Don't write new cache (current context already cached)
    // Pass compaction context's abortController: user Esc can interrupt
  })
  // Extract summary text from result messages
}
```

Why pass parent session's cacheSafeParams: The compaction fork needs to use exactly the same cache key as the main session (system prompt + tools + message prefix) to hit the prompt cache already created by the main session. If the cache key doesn't match, the fork's cache hit rate is extremely low (~98% miss for 3P users).

> **Note**
>
> The compaction system prompt remains unchanged -> but shouldn't KV cache also miss after images and other context are stripped?

```
所以子 agent 收到的完整请求是:

系统提示词 (继承) : "You are a helpful AI assistant..."
    ↓
工具定义 (继承) : Read, Edit, Bash, ...
    ↓
对话历史 (继承) : [所有压缩前的消息]
    ↓
用户消息 (摘要请求) : "请按以下格式总结上述对话: ..."
```

	Fork 路径	直接流式路径
系统提示词	**父进程的** (coding assistant)	**专用的** (summarizing)
能否复用 KV Cache	✅ 能 (系统提示词一样)	❌ 不能 (提示词变了)
摘要能力来源	user message 指令 + canUseTool 禁止	系统提示词直接定义

7.9 Direct Streaming Path (Fallback)

When the fork path fails or cache sharing is disabled, use direct streaming call:

```
// Tool list: only FileReadTool (+ ToolSearchTool if enabled)
// Key: do not set maxOutputTokens (avoid mismatching parent session cache key)
// Alternative: set in options.maxOutputTokensOverride
```

> **Note**
>
> - During compaction, directly call API in the current process, receiving summary via streaming
> - **At this time there is a dedicated summary system prompt, different from the parent process** -> ==essentially cannot hit KV cache anyway

---

8. Session Memory

Session Memory is a **preventive** context management mechanism. It continuously maintains a Markdown note in the background during conversation, **extracting key information**. This allows compaction to replace the entire conversation history with "session notes + preserved tail messages" while retaining substantial context.

8.1 Core Difference from Compaction

	Full Compaction	Session Memory
Timing	When context pressure is too high	Continuously maintained during conversation
Method	Generate summary from scratch	**Incrementally update** existing notes
Content	Regenerated each compaction	Gradually accumulated
API Call	Required	Required (**Sub-agent updates**)
Info Coverage	Depends on model summary quality	Incremental accumulation, more comprehensive

8.2 Trigger Conditions

```
// sessionMemoryUtils.ts
DEFAULT_SESSION_MEMORY_CONFIG = {
  minimumMessageTokensToInit: 10_000,    // Init: context must be at least 10K tokens
  minimumTokensBetweenUpdate: 5_000,     // Update: must grow by at least 5K tokens
  toolCallsBetweenUpdates: 3,            // Update: at least 3 tool calls
}
```

Trigger conditions (must **all be met**):

```
shouldExtractMemory(messages: Message[]): boolean {
  // 1. Already initialized (minimumMessageTokensToInit already met)
  // 2. Token growth >= minimumTokensBetweenUpdate
  // 3. Tool call count >= toolCallsBetweenUpdates
  // 4. Last assistant message has no tool calls (natural conversation pause point)

  const shouldExtract =
    (hasMetTokenThreshold && hasMetToolCallThreshold) ||
```

```
    (hasMetTokenThreshold && !hasToolCallsInLastTurn)
}
```

Why require "no tool calls in the end": This ensures extraction happens during a natural conversation gap, not mid-tool-call, avoiding missed context.

8.3 Sub-Agent Execution Flow

```
Main conversation loop (after each response)
    ↓
postSamplingHook: extractSessionMemory
    ↓
Check trigger conditions (shouldExtractMemory)
    ↓
setupSessionMemoryFile()
    ├─ Create ~/.claude/sessions/<id>/session-memory.md (if not exists)
    ├─ Initialize using template
    └─ FileReadTool reads current content
    ↓
buildSessionMemoryUpdatePrompt(currentMemory, memoryPath)
    ├─ Load custom prompt.md or use default
    ├─ Analyze each section's token size
    └─ Generate warning if exceeding limit
    ↓
runForkedAgent({
  promptMessages: [userPrompt],
  canUseTool: createMemoryFileCanUseTool(memoryPath),  // Only allow Edit this file
  querySource: 'session_memory',
  forkLabel: 'session_memory',
  maxTurns: 1,    // Usually completes in 1 turn (model directly edits)
})
    ↓
Record token usage
Update lastSummarizedMessageId
```

> **Note**
>
> The session memory sub-agent reuses the main agent's system prompt, meaning KV can hit; and Claude supports multiple cache marker positions

8.4 Session Memory Template Explained

```
# Session Title
_A short, recognizable 5-10 word title_
(Example: Optimize PostgreSQL query performance)

# Current State
_What is currently being worked on? Unfinished pending tasks. Next steps._
(This is the most critical section; after compaction, this is the first thing to check)

# Task specification
_What is the user asking to build? Design decisions and other context._
```

```
# Files and Functions
_What are the important files? Brief descriptions of their content and relevance._
(Include file paths, function signatures, key logic)

# Workflow
_Common bash commands and their order? How to interpret output?_
(Deployment process, test commands, startup commands, etc.)

# Errors & Corrections
_Encountered errors and fixes. What did the user correct? What approaches failed?_
(Include error messages, fix steps, user feedback)

# Codebase and System Documentation
_Important system components? How they work/combine?_

# Learnings
_What worked well? What didn't? What to avoid?_

# Key results
_Specific outputs requested by user (answers, tables, documents). Repeat exactly._
(Preserve complete tables, code snippets, answers)

# Worklog
_Brief log of each step._
(Concise timeline for review)
```

Note

From practical experience, the following sections can be added:

```
# Global Constraints & Preferences
_Behavioral boundaries and absolute don'ts_
(Example: No direct main branch modifications, output must strictly follow a specific
style)

# Environment Context
Current execution task's specific system and external environment context
(Includes database connection string format, specific port numbers, system and key
dependency versions)
```

8.5 Token Limits

```
// sessionMemory/prompts.ts
MAX_SECTION_LENGTH = 2000          // Max 2000 tokens per section
MAX_TOTAL_SESSION_MEMORY_TOKENS = 12000  // File total max 12K tokens

// When exceeding limits:
  // 1. Oversized section: generate "this section exceeds limit, please condense" warning
// When compressing:
  // 2. Total tokens over limit: add "File already exceeds 12K tokens, please condense
```

```
//     all sections" at the end of the prompt
// Compression uses truncateSessionMemoryForCompact():
//    Max 8000 chars per section preserved (MAX_SECTION_LENGTH × 4)
//    Truncate overflow and add "[content truncated]" marker
```

Note

Scenario 1: Sub-agent updates session memory

Let the sub-agent condense itself; the sub-agent receives a prompt with an additional section at the end:

```
Oversized sections to condense:
- "# Errors & Corrections" is ~3500 tokens (limit: 2000)
- "# Key results" is ~2800 tokens (limit: 2000)

CRITICAL: The session memory file is currently ~14000 tokens,
which exceeds the maximum of 12000 tokens.
```

Scenario 2: Injecting session memory into messages during compaction

Call truncateSessionMemoryForCompact → directly truncate via function, not through sub-agent

```
Per-section char limit = MAX_SECTION_LENGTH × 4 = 2000 × 4 = 8000 chars
                    ↑ roughTokenCountEstimation uses length/4
                    so char limit = 2000 tokens × 4 chars/token

Processing:
  1. Split each section by '# '
  2. For each section, keep lines from the beginning
  3. Keep until char limit is reached
  4. Add truncation marker

Original content (a section):
  # Errors & Corrections
  _Encountered errors and fixes..._
  - Error1: cause, fix
  - Error2: cause, fix
  - Error3: cause, fix  ← exceeds limit
  - Error4: cause, fix  ← truncated
  - Error5: cause, fix  ← truncated

After truncation:
  # Errors & Corrections
  _Encountered errors and fixes..._
  - Error1: cause, fix
  - Error2: cause, fix
  - Error3: cause, fix
  [... content truncated; use Read on the skill path if you need the full text]
  ↑ Truncation marker, telling model it can re-read the skill file
```

8.6 Custom Template Support

```
// ~/.claude/session-memory/config/template.md  → session memory template
// ~/.claude/session-memory/config/prompt.md    → update prompt
// Custom prompt uses {{currentNotes}} and {{notesPath}} variables
```

8.7 Tool Usage Restrictions

```
createMemoryFileCanUseTool(memoryPath): CanUseToolFn
// Only allow: Edit tool, and only if file_path === memoryPath
// Deny: all other tools
// This is a highly restricted environment, preventing sub-agent from unexpected actions
```

8.8 Mutual Exclusion with Main Agent

Mutual exclusion rules between session memory sub-agent and main agent:

- **Main agent writes**: Skip sub-agent's extraction this round, advance cursor
- **Sub-agent writes**: Main agent won't write again
- **Simultaneous write attempts**: Cannot happen, as they are in different processes

8.9 Session Memory Usage

The main agent does not actively read this file. It **is only used during compaction**

```
autoCompact / manual compact
    └── trySessionMemoryCompaction()
          ├── getSessionMemoryContent()         ← Read file directly
          ├── isSessionMemoryEmpty()           ← Check for real content
          ├── calculateMessagesToKeepIndex()   ← Calculate preservation range
          │                                      based on lastSummarizedMessageId
          └── createCompactionResultFromSessionMemory()
                ├── truncateSessionMemoryForCompact()   ← Truncate long content
                ├── getCompactUserSummaryMessage()       ← Wrap content as
                │                                          summary user message
                └── buildPostCompactMessages()          ← Add to message history

  After compaction, session memory file content enters the main agent's
  messages list via the summary user message, becoming part of the
  conversation context indirectly
```

---

9. Session Memory Compaction (Important)

Session Memory Compaction (SM-Compact) is an **alternative to** full compaction, using incrementally accumulated session notes instead of summaries generated from scratch each time.

9.1 Comparison with Traditional Compaction

Dimension	Traditional Compaction	SM-Compact
Summary source	Generated from scratch each time	Incrementally maintained session notes
History handling	All replaced with summary	Selectively preserve tail
API call	Required (model generates summary)	**Not required**
Content preserved	Summary text only	Summary + most recent 10K-40K tokens

Dimension	Traditional Compaction	SM-Compact
Use case	General	Long multi-turn sessions

> **Note**
>
> **SM-Compact has higher priority than traditional compaction**; falls back to traditional compaction on failure

SM-Compact advantages:

```
  Traditional compactConversation requires each time:
  fork agent → call summarization API → wait for response → generate summary

  SM-Compact only requires:
  Directly read summary.md file → generate summary user message → return

  No API call, zero latency, zero cost
```

9.2 Preserved Message Calculation

```
// services/compact/sessionMemoryCompact.ts
calculateMessagesToKeepIndex(messages, lastSummarizedIndex): number

// From lastSummarizedMessageId, expand backward:
  // 1. Satisfy minTokens (default 10K)
  // 2. Satisfy minTextBlockMessages (default 5)
  // 3. Don't exceed maxTokens (default 40K)
  // 4. floor = last compaction boundary + 1 (don't cut within compaction boundary)
```

`adjustIndexToPreserveAPIInvariants` **Protection**:

API has two hard constraints that compaction must satisfy:

Constraint 1: Don't split tool_use/tool_result pairs

Constraint 2: Don't split streaming messages sharing message.id

> **Note**
>
> This is due to API behavior in streaming mode. Specifically, in streaming mode, one assistant response arrives in segments. Therefore, if any block of a message.id is preserved, all blocks of that message.id must be preserved — because the API ultimately merges by message.id

```
Claude's streaming emits a separate assistant message for each content block:
  Index N:   assistant(id=X), content: [thinking]      ← streaming chunk 1
  Index N+1: assistant(id=X), content: [tool_use: ID]  ← streaming chunk 2
  Index N+2: user, content: [tool_result: ID]

If startIndex = N+1 (only keep tool_use but discard thinking):
  → normalizeMessagesForAPI merges by message.id
  → thinking block is lost
  → model loses important reasoning process
```

```
Correct approach:
  → Detect that N+1 shares message.id with N
  → Adjust startIndex to N (include complete streaming group)
```

9.3 Feature Usage Determination

Trigger condition: `effective context window × 90%` (traditional compaction trigger is effective context window × 93.5%)

Multi-layer progressive check for whether SM-Compact can be used:

```
shouldUseSessionMemoryCompaction(): boolean
// 1. `tengu_session_memory` feature flag = true; main switch: whether session memory
//    feature itself is enabled
// 2. `tengu_sm_compact` feature flag = true; sub-switch: whether session memory driven
//    compaction is enabled
// 3. Check env vars (feature switches): ENABLE_CLAUDE_CODE_SM_COMPACT (force enable)
//    or DISABLE_CLAUDE_CODE_SM_COMPACT (force disable)
// 4. Session memory file exists and non-empty (isSessionMemoryEmpty())
```

9.4 Execution Flow

```
trySessionMemoryCompaction(messages, agentId, autoCompactThreshold)
    ↓
1. Wait for any in-progress session memory extraction (15s timeout)
    ↓
2. Get session memory file content
    ↓
3. Locate lastSummarizedMessageId (try to recover if missing)
    ↓
4. Calculate message range to preserve (calculateMessagesToKeepIndex)
    ↓
5. Filter out old compaction boundary messages
    ↓
6. Execute SessionStart Hooks (reload CLAUDE.md)
    ↓
7. Create compaction result
    ├─ createCompactBoundaryMessage()
    ├─ Convert session memory content to summary message
    ├─ truncateSessionMemoryForCompact() (ensure within budget)
    └─ Attach plan etc.
    ↓
8. Check postCompactTokenCount < autoCompactThreshold
    ↓ Pass → return result
    ↓ Fail → return null, fall back to traditional compaction
```

9.5 Compaction Boundary Metadata

```
// compact.ts
annotateBoundaryWithPreservedSegment(
  boundary,
  anchorUuid,   // Anchor: last summary message
```

```
  messagesToKeep,
): SystemCompactBoundaryMessage

// preservedSegment tells session storage how to restore chain structure:
// Points to original messages in physical storage, used to tell session storage
// how to rebuild the association before and after compaction

// headUuid → first of preserved segment; UUID of oldest message replaced
// anchorUuid → summary message (mount point); UUID of new summary message
//              generated by summarizer
// tailUuid → last of preserved segment; UUID of last message in preserved segment
```

---

10. Auto-Compact

Auto-Compact is a compaction mechanism that automatically triggers when context approaches the threshold, without user manually executing `/compact`.

> **Note**
>
> However, for reactive compact, no specific implementation code is provided

10.1 Threshold System Explained

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Effective Context Window                        │
│                                                                          │
│  0%        warning          autocompact        blocking     100%        │
│   ├──────────┤├──────────────┤├─────────────────┤├────────────┤          │
│   0         180K           187K              197K        200K          │
│             ↑              ↑                ↑                          │
│      isAboveWarning    shouldAutoCompact  isAtBlocking                  │
│       Threshold           → Auto-compact    Limit → Block input        │
│                                                                          │
│  WARNING_BUFFER = 20K    AUTO_BUFFER = 13K  MANUAL_BUFFER = 3K          │
└──────────────────────────────────────────────────────────────────────────┘
```

10.2 Warning State

```
calculateTokenWarningState(tokenUsage, model): {
  percentLeft: number,                    // Remaining space percentage
  isAboveWarningThreshold: boolean,       // Issue warning (yellow)
  isAboveErrorThreshold: boolean,         // Issue error (red)
  isAboveAutoCompactThreshold: boolean,  // Trigger auto-compact
  isAtBlockingLimit: boolean,            // Block input
}
```

10.3 Compaction Priority

```
autoCompactIfNeeded(messages, toolUseContext, cacheSafeParams, ...): {
  wasCompacted: boolean,
```

```
  compactionResult?: CompactionResult,
  consecutiveFailures?: number,
}

// Execution order:
  // 1. Check circuit breaker (consecutive failures >= 3 → return)
  // 2. shouldAutoCompact() → check threshold
  // 3. Try SM-Compact (trySessionMemoryCompaction)
  //    → Success → return
  // 4. Execute traditional compaction (compactConversation)
  //    → Success → reset circuit breaker consecutiveFailures = 0
  //    → Fail → consecutiveFailures++
```

10.4 Circuit Breaker Explained

Circuit breaks after three consecutive failures

```
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3

// Data background (2026-03-10):
// 1,279 sessions had 50+ consecutive failures (max 3,272 times)
// ~250K wasted API calls per day

// On consecutive failures:
  // - Skip futile compaction attempts
  // - User receives "conversation is full" error, needs manual handling
  //   (/clear or /compact with options)
  // - Next new input re-checks threshold
```

10.5 Disable Control Matrix

Control Method	Effect	Priority
`DISABLE_COMPACT=1`	Disable all compaction	Strongest
`DISABLE_AUTO_COMPACT=1`	Disable auto-compact only, keep `/compact`	Strong
`userConfig.autoCompactEnabled=false`	User config	Medium
`CONTEXT_COLLAPSE` feature	Collapse system takes over, disable auto-compact	Medium
`REACTIVE_COMPACT` feature	Disable proactive compaction, rely on 413 response	Medium

10.6 Marble Origami Special Case

When `CONTEXT_COLLAPSE` is on, marble_origami (ctx-agent)'s auto-compact is disabled:

- If ctx-agent context blows up, auto-compact triggers
- `runPostCompactCleanup` calls `resetContextCollapse()`
- This would corrupt the **main thread**'s committed log (shared module state)

Solution: When `CONTEXT_COLLAPSE` is on, ctx-agent disables auto-compact.

> **Note**
>
> - marble_origami is a ctx-agent (context collapse agent), running in a sub-agent/fork. It shares same-process module-level state with the main thread

- marble_origami is a shared-process-state ctx-agent; if its auto compact succeeds, it triggers resetContextCollapse() — even with an isMainThreadCompact check, the shared state existence itself is unsafe. So directly exclude marble_origami at the shouldAutoCompact source; ctx-agent's context explosion is handled by the Context Collapse system itself, not the auto compact path

---

11. Context Collapse (Experimental Feature; all content is speculative)

Context Collapse (`CONTEXT_COLLAPSE` feature flag) is an **experimental** context management mechanism controlled via feature gates.

> **Note**
>
> - Context Collapse is an already-implemented, currently in internal灰度 testing experimental feature, not yet officially released. It has no external documentation or stable product naming
> - Since the core service source code (services/contextCollapse/) is not included in restored-src/, only cross-analysis of 14 reference points is used to infer its complete logic

11.1 Core Difference from Compaction

```
Compaction:
  Discard old messages → replace with summary
  Information loss: Medium (controlled by prompt quality)
  Latency: High (requires AI model call to generate summary)
  Precision: Low-Medium (summary quality unstable)

Context Collapse:
  Selective discard → precise control over preserved content
  Information loss: Precisely controllable
  Latency: Low (no AI call needed)
  Precision: High (precise proportional discard)
```

11.2 Design Principles

Similar to operating system memory management paging strategy:

- **90% threshold**: Begin commit operation — write intermediate results to disk
- **95% threshold**: Block new input — wait for commit to complete
- **Preservation strategy**: Keep the most recent N% of messages, ensuring working context is not lost

11.3 Mutual Exclusion with Auto-Compact

Priority order: **Context Collapse > Session Memory Compact > Auto-Compact**

```
When CONTEXT_COLLAPSE is on:
  - Auto-Compact is fully suppressed (avoid competition)
  - 90% commit threshold < 93.5% auto-compact threshold
  - Collapse system takes over before auto-compact
  - 95% blocking threshold > 93.5% auto-compact threshold

When CONTEXT_COLLAPSE is off:
```

```
  - Auto-Compact operates normally
  - Relies on summarization to manage context
```

11.4 Implementation Location

```
// Dynamic import (avoid circular dependency):
if (feature('CONTEXT_COLLAPSE')) {
  const {
    isContextCollapseEnabled,
    applyCollapsesIfNeeded,
    recoverFromOverflow,
    isWithheldPromptTooLong,
    resetContextCollapse,
  } = require('../services/contextCollapse/index.js')
}
```

Why dynamic import: This module is in the `services/contextCollapse/` directory, which is not included in the source map restoration file list. This indicates `contextCollapse` is a compiled standalone module, loaded via `require()` with lazy loading.

11.5 Objects Being Collapsed

Operates on **local context** for each message span, so the idea is still collapsing; specifically:

```
  user message → assistant message → user message → ... → assistant message
   ↑                                                            ↑
   firstArchivedUuid (span start)                          lastArchivedUuid (span end)
```

The product of collapsing: each collapsed span is replaced with a metadata placeholder:

```
<collapsed id="16-char-collapseId">Summary text</collapsed>
```

11.6 Two-Phase Collapse Algorithm

Context Collapse is **preventive and progressive**

Phase 1 — Staging (Preliminary Collapse)

Trigger condition: 90% effective context window
effectiveWindow tokens ≥ effectiveWindow × 90%

Triggers:

1. ctx-agent spawn (sub-process, codename "marble_origami") is started
2. ctx-agent analyzes all message spans from "protected tail boundary" to current position
3. **Generates summary for each span**: { startUuid, endUuid, summary, risk, stagedAt }
4. Results written to staged queue (in memory)
5. Writes a marble-origami-snapshot entry to transcript (persist staged queue state)
6. Staged spans are not immediately deleted from API context, just waiting in queue

> Significance of staging: Prepare summaries ahead of time, don't wait until overflow

Phase 2 — Commit (Formal Collapse)

Trigger conditions have two paths:

Path A (Proactive) — 95% blocking trigger
effectiveWindow tokens ≥ effectiveWindow × 95%
Before the next spawn request is sent, check if already staged:

- If staged → commit directly (use existing summary, no model call needed again)
- If not staged → generate summary immediately then commit

Path B (Reactive) — 413 overflow recovery
API returns 413 (prompt too long)
Call recoverFromOverflow():

1. Commit all spans in staged queue (drain)
2. Retry API request with new context after commit
3. If retry gets 413 again → fall back to reactiveCompact

11.7 Relationship with Other Mechanisms

```
用户输入 + 完整历史消息
        │
        ▼
  ① microcompact
        │ tool_use_id 去重 + 移除 cache metadata
        │ (只处理格式，不减少 token 数)
        ▼
  ② snipCompact(可选，HISTORY_SNIP 开关)
        │ 截断超长 tool result
        │ tokensFreed 会传给 autocompact 的阈值计算
        ▼
  ③ Context Collapse — applyCollapsesIfNeeded()
        │ 核心逻辑：staging + commit
        │ 仅当 feature('CONTEXT_COLLAPSE') 开启
        ▼
  ④ autocompact
        │ ⚠ 当 Context Collapse 开启时 → 完全跳过 (shouldAutoCompact = false)
        │ 当 Context Collapse 关闭时 → 正常触发 (~93% 阈值)
        ▼
  ⑤ API调用 (callModel)
```

机制	触发时机	效果	持久化	优先级
microcompact	始终（每轮）	格式去重，不删内容	否	最先
snipCompact	HISTORY_SNIP 开启时	截断长 tool result	boundary 消息	其次
Context Collapse	90% staging / 95% commit	span → placeholder	commit + snapshot 条目	第三
autocompact	~93%，CC 关闭时	批量 summary	summary 消息	CC 开启时跳过
reactiveCompact	CC 兜底后 / RC 单独启用	全量 summary	summary 消息	最后兜底

```
isContextCollapseEnabled() ?
│
├── FALSE → autocompact 正常检查 (~93%)
│              └── reactiveCompact 作为 413 fallback
│
└── TRUE
    │
    ├── autocompact 完全跳过 (shouldAutoCompact = false)
    │
    ├── API 调用前：applyCollapsesIfNeeded()
    │    │
    │    ├── tokens < 90%  → no-op
    │    ├── tokens ≥ 90%  → spawn ctx-agent, stage spans
    │    └── tokens ≥ 95%  → commit staged spans
    │
    └── API 返回 413:
         │
         ├── recoverFromOverflow()
         │    └── drain → retry → 还 413?
         │          │
         │          └── reactiveCompact (兜底)
         │
         └── 如果 reactiveCompact 也无 → 暴露错误
```

---

12. Context Analysis

12.1 Use Cases

`analyzeContext()` scans the messages array and generates detailed statistics on token consumption, used for:

1. **Telemetry reporting**: Report detailed dimensions of `tengu_compact` events via Statsig
2. **Diagnostic analysis**: Identify optimizable waste points (e.g., duplicate file reads)

> **Note**
>
> - analyzeContextUsage() is not just for token consumption statistics; it's a complete context audit system — responsible for sensing, classifying, quantifying, and driving actionable optimization suggestions
> - `tengu` is the Anthropic internal project codename prefix, spanning the entire codebase (1051 references, 250 files). In Japanese folklore, Tengu is an omnipotent spirit. This codename suggests how important this project is to Anthropic internally
> - "Identifying optimizable waste points" is purely regularized statistics + hardcoded heuristic rules, no LLM calls

12.2 Statistical Dimensions

```
type TokenStats = {
  toolRequests: Map<string, number>      // Token consumption by tool request
  toolResults: Map<string, number>       // Token consumption by tool result
  humanMessages: number                   // Total user message tokens
  assistantMessages: number              // Total assistant message tokens
  localCommandOutputs: number            // Bash command output tokens
  other: number                          // Other (thinking, attachments, etc.)
  attachments: Map<string, number>         // Count by attachment type
  duplicateFileReads: Map<string, {       // Duplicate file reads
    count: number
    tokens: number
  }>
  total: number                          // Total tokens
}
```

12.3 Duplicate Read Detection Algorithm

```
// For multiple reads of the same file, count only once
// Duplicate tokens = avgTokensPerRead × (count - 1)

fileReadStats.forEach((data, path) => {
  if (data.count > 1) {
    const avgTokensPerRead = Math.floor(data.totalTokens / data.count)
    const duplicateTokens = avgTokensPerRead * (data.count - 1)
    stats.duplicateFileReads.set(path, { count, tokens: duplicateTokens })
  }
})
```

Practical application: If a file was read 10 times, 1K tokens each time, then `duplicate_read_tokens` = 9K. This means after compaction, the model doesn't need to re-read this file, saving substantial tokens.

12.4 Statsig Metric Mapping

```
tokenStatsToStatsigMetrics(stats): Record<string, number>

// Generates 40+ metrics, including:
// - Token consumption and percentage by tool
// - Duplicate read tokens and file count
// - Token distribution by message type
// - Attachment type counts
```

固定 Token 指标（5个）

指标名	含义
total_tokens	消息总 token 数（不含 system prompt / tools）
human_message_tokens	用户消息的 token 数（含图片被 strip 成 [image] 后的 text）
assistant_message_tokens	助手消息的 token 数（含 thinking、tool_use block 的文本表示）
local_command_output_tokens	本地命令输出 token（local-command-stdout 标记的内容，通常是 BashTool 执行结果的文本）
other_tokens	其他所有块类型的 token（thinking、search_result、document、compaction 等）

附件计数（n个，每种类型一个）

指标名	含义
attachment_${type}_count	该类型附件出现的次数

常见类型：
- attachment_file_count — 文件附件（createFileAttachment / post-compact 恢复的文件）
- attachment_skill_discovery_count — 技能发现信号
- attachment_skill_listing_count — 技能列表
- attachment_command_permissions_count — 命令权限附件
- attachment_invoked_skills_count — 被调用的 skill 附件
- attachment_plan_mode_count — plan mode 附件

工具请求 Token（n×2个，每工具）

指标名	含义
tool_request_${tool}_tokens	该工具被调用时，tool_use block 本身的 token 数（包含工具名、参数名/值的序列化文本）
tool_result_${tool}_tokens	该工具返回结果，tool_result block 的 token 数

```
tool_request_Read_tokens      → Read tool 调用时发出的参数（file_path 等）
tool_result_Read_tokens       → Read tool 返回的文件内容 token
tool_request_Bash_tokens      → Bash 调用参数
tool_result_Bash_tokens       → Bash 执行结果输出 token
tool_request_Edit_tokens      → Edit 参数
tool_result_Edit_tokens       → Edit 结果
...
```

重复文件读取（2个）

指标名	含义
duplicate_read_tokens	所有被重复读取的文件的浪费 token 总数。计算方式：avg_tokens_per_read × (count - 1)（每次重复读取浪费 1 倍平均大小）
duplicate_read_file_count	出现了重复读取的文件种类数（有多少个不同文件被读了多次）

注意：duplicate_read_tokens 是**浪费量的估算**，不是实际消耗。

百分比指标（6个固定 + n×2个动态）

指标名	含义
human_message_percent	用户消息 token 占总消息 token 的百分比
assistant_message_percent	助手消息 token 占比
local_command_output_percent	本地命令输出 token 占比
tool_request_percent	所有 tool_request block 总 token 占比
tool_result_percent	所有 tool_result block 总 token 占比
duplicate_read_percent	重复读取浪费 token 占总 token 的百分比
tool_request_${tool}_percent	该工具的 request token 占总 token 的百分比
tool_result_${tool}_percent	该工具的 result token 占总 token 的百分比

百分比都是四舍五入到整数。

Categories overview

```
  ContextData.categories = [
    { name: 'System prompt',        tokens: N },   // system prompt sections breakdown
    { name: 'System tools',        tokens: N },   // built-in tool schemas (ANT users can
see each tool detail)
    { name: 'MCP tools',           tokens: N },   // MCP server tools (deduplicated)
    { name: 'MCP tools (deferred)', tokens: N, isDeferred: true },  // loaded but not
consuming
    { name: 'System tools (deferred)', tokens: N, isDeferred: true },
    { name: 'Custom agents',       tokens: N },
    { name: 'Memory files',        tokens: N },   // CLAUDE.md per-file breakdown
    { name: 'Skills',              tokens: N },   // skill frontmatter (excluding full
content)
    { name: 'Messages',            tokens: N },   // conversation history (after
microcompact)
    { name: 'Autocompact buffer',   tokens: N },   // safety space reserved for autocompact
(skipped when CC on)
    { name: 'Free space',          tokens: N },
  ]
```

Key: categories with `isDeferred: true` are not counted in actualUsage — they represent "loaded but token-consuming only on demand"

Deferred Tools:

- Only tell model the tool name, not the full schema; load on demand
- Important to note: MCP tools configured by users are the main target: **MCP tools are deferred by default** (isMcp === true takes that branch), while built-in tools are only deferred if explicitly marked shouldDefer (some built-in tools are prohibited from deferral)
- Normally, each API request includes all tools' complete schemas (name + description + parameters) in the request header. The Deferred mechanism changes this:

```
  Regular tools (non-deferred)
    → API request includes complete schema (token consumption immediate)

  Deferred tools
    → API request only has tool name (minimal tokens)
```

```
    → When model wants to call → first call ToolSearchTool to pull full schema → then
execute
```

The idea is similar to progressive disclosure of skills; even the code directly states this:

```
// loadSkillsDir.ts:97
/**
 * Estimates token count for a skill based on frontmatter only
 * (name, description, whenToUse) since full content is
 * only loaded on invocation.
 */
export function estimateSkillFrontmatterTokens(skill: Command): number
```

Skill 的渐进式披露（对比 Deferred Tool）

	Skill	Deferred Tool
披露时机	首次被调用时	首次被调用时
初始加载	name + description + whenToUse (frontmatter)	只有工具名
完整内容	--- 分隔符后的 markdown 全文	完整 JSONSchema
触发方式	模型调用 SkillTool	模型调用 ToolSearchTool
加载后	指令直接执行，不重复调用	schema 进入可用工具集

统一的设计模式：按需加载（Lazy Load）

```
Skills:
  初始：frontmatter token 数 → 模型感知到 N 个 skill 存在
  调用：SkillTool → 完整 markdown 加载 → 执行指令

Deferred Tools:
  初始：工具名字典 → 模型感知到 N 个工具存在
  调用：ToolSearchTool → 完整 schema 加载 → 可调用

Deferred MCP Tools (always deferred) :
  初始：mcp__server__toolname 只有名字
  调用：ToolSearchTool → 完整 schema → 执行
```

共同解决的问题

```
问题：工具/skill 全集太大，全部塞进上下文太贵
解法：不塞全集，只塞"目录"（名字 + 简介）

代价：第一次调用有额外延迟（需要拉取完整内容）
收益：未使用的部分完全不消耗 token
```

12.5 Detecting Optimizable Waste Points

Waste detection has two systems:

System A: generateContextSuggestions() — User-facing suggestions

检查函数	检测条件	优化建议
checkNearCapacity	percentage ≥ 80%	"建议现在手动 /compact，否则 autocompact 会自动丢失消息"
checkLargeToolResults	某 tool 的 call+result > 15% 上下文 或 > 10k tokens	"工具 X 的结果太大，考虑分流或截断"
checkReadResultBloat	Read tool results > 5% 上下文	"Read 结果太冗余，考虑只读必要的部分"
checkMemoryBloat	CLAUDE.md > 5% 上下文 或 > 5k tokens	"CLAUDE.md 太大，考虑精简"
checkAutoCompactDisabled	autocompact 关闭 且上下文 > 80%	"autocompact 关闭，手动 /compact 来释放空间"

System B: checkContextWarnings() — doctor command warnings

检查函数	检测条件	警告
checkClaudeMdFiles	任意 CLAUDE.md 文件 > 40k chars	列出每个大文件及字符数
checkAgentDescriptions	agent 描述 token > 阈值	列出每个 agent 的估算 token
checkMcpTools	MCP tools 总 token > 25k	按 server 分组列出 token 分布
checkUnreachableRules	权限规则被其他规则遮蔽	列出每个失效规则 + 修复建议

12.6 Summary

User-facing UI layer
/context command ← ContextData rendering
ContextVisualization ← gridRows + categories rendering
TokenWarning ← percentage-triggered warning

Suggestion layer
contextSuggestions ← Proactive suggestions (what can be optimized)
doctorContextWarnings ← Warnings (config/scale issues)

Statistics layer
analyzeContextUsage ← Bucket counting, API/client dual precision:
When API returns actual usage, use server numbers; otherwise use client estimate
← deferred mechanism (saves tokens)
← per-tool granularity breakdown, "which tool costs most" stats,
message distribution by tool type
← counted after microcompact, ensures stats reflect tokens
the model actually sees

---

13. Post-Compaction Cleanup

`runPostCompactCleanup()` executes after each compaction (auto/manual), cleaning up state polluted by the compaction operation.

13.1 Cleanup Contents Explained

```
runPostCompactCleanup(querySource?: QuerySource): void

// Cleanup executed only for main thread compaction:
resetMicrocompactState()              // Reset microcompact state (tool registration, cache
edit queue)
getUserContext.cache.clear()          // Clear user context cache (CLAUDE.md content)
resetGetMemoryFilesCache()            // Clear in-memory file discovery cache
clearSystemPromptSections()           // Clear system prompt section cache
clearClassifierApprovals()           // Clear classifier approvals (avoid approvals
forgotten)
clearSpeculativeChecks()              // Clear speculative check state
clearBetaTracingState()               // Clear beta tracing state
clearSessionMessagesCache()           // Clear session message cache
sweepFileContentCache()               // Clear post-hoc attribution file cache
(COMMIT_ATTRIBUTION)

// Context Collapse (main thread only):
if (feature('CONTEXT_COLLAPSE') && isMainThreadCompact) {
  resetContextCollapse()              // Reset context collapse state
}
```

13.2 Key Items NOT Cleared

```
// The following are intentionally NOT cleared (persist across compactions):
// 1. invoked_skills → needed by createSkillAttachmentIfNeeded
// 2. sentSkillNames → reinjecting complete listing is pure waste
```

> **Note**
>
> invoked_skills records skills that have already been executed; the skill currently being executed is not in invoked_skills yet. So "partial execution" is not an issue
> Then what does it matter that the model knows which skills have been used?
> The explanation given by the LLM is that it preserves the actual content of skills that have proven valuable.
> But this reasoning seems to have holes... skills that have been executed may not necessarily need to be executed again later

13.3 Main Thread vs. Sub-Agent Distinction

```
// Sub-agents (agent:*, sdk) also call functions in compact.ts
// But their compaction should not clear main thread's cache
const isMainThreadCompact =
  querySource === undefined ||
  querySource.startsWith('repl_main_thread') ||
  querySource === 'sdk'
```

Sub-agent post-compaction cleanup:

```
压缩发生在主线程？
  ├── 是 → 清理所有状态 (microcompact + 共享状态)
  └── 否 (子 agent) → 不调用
```

因为下面这些是**主线程共享的模块级状态**，子 agent 乱清会破坏主线程：

状态	为什么子 agent 不能清
resetContextCollapse()	CC commit log 是主线程的，清了主线程的折叠全丢了
clearSessionMessagesCache()	主线程的 transcript 缓存，清了 UI 会看到 stale 数据
getUserContext.cache.clear()	CLAUDE.md 是主线程的，子 agent 清理等于帮倒忙
clearSystemPromptSections()	system prompt sections 是主线程的渲染状态
sweepFileContentCache()	归因文件缓存是主线程的

而下面这些是**子 agent 自己隔离的**，所以子 agent 自己清：

状态	为什么子 agent 可以自己清
resetMicrocompactState()	子 agent 有自己的工具注册表和缓存编辑队列，不清会内存泄漏
contentReplacementState	子 agent 有自己的替换状态映射，与主线程隔离

Summary:
After sub-agent compaction:
✅ resetMicrocompactState() — clear its own (isolated)
✅ contentReplacementState reset — clear its own (isolated)
❌ runPostCompactCleanup() — not called (won't mistakenly clear main thread shared state)

After main thread compaction:
✅ runPostCompactCleanup() — all cleanup items execute

Core principle: whoever compacts clears their own isolated state; whoever calls runPostCompactCleanup clears shared state. Sub-agent compaction is isolated; it only responsible for itself.

---

14. Context Loading and Caching

14.1 System Context (`context.ts`)

> **Note**
>
> Not the system prompt itself, but extra information injected into the system prompt

```
注入后的 system prompt 结构

You are Claude Code...

You have access to tools: Read, Edit, Bash...

[环境信息 - systemContext]
Current branch: main
Status: (clean)
Recent commits: a8a678c init...

[用户配置 - userContext]
Today's date is 2026-04-02.

# CLAUDE.md 内容（从项目根目录加载）

[对话历史 - messages]
...
```

Git Status

```
getGitStatus(): Promise<string | null>
// Contains:
  // - Current branch
  // - Main branch
  // - Git username
  // - git status --short (max 2000 chars, truncate if exceeded)
  // - Most recent 5 commits

// Caching: valid for entire session (memoized)
// Skip conditions:
  // 1. NODE_ENV=test (avoid test loops)
  // 2. Not a git directory (getIsGit() === false)
  // 3. CLAUDE_CODE_REMOTE env var (CCR mode, skip to save cost)
  // 4. Git instructions disabled (shouldIncludeGitInstructions())
```

User Context

```
getUserContext(): Promise<{ claudeMd, currentDate }>
// claudeMd: content loaded from all CLAUDE.md and memory files
// currentDate: current ISO date

// Caching: valid for entire session (memoized)
// Disable conditions:
  // 1. CLAUDE_CODE_DISABLE_CLAUDE_MDS=1 (hard disable)
  // 2. --bare mode and no explicit --add-dir (skip auto-discovery)
```

Cache Invalidation

```
// When context needs to be reloaded (e.g., user modified CLAUDE.md):
setSystemPromptInjection(value)  // Set cache breaker
// → getUserContext.cache.clear()
```

```
// → getSystemContext.cache.clear()
// → reload on next call
```

> **Note**
>
> CLAUDE.md loads at three moments
>
> - Moment 1: At session start
> - Moment 2: After compaction (auto-reload)
> - Moment 3: When user modifies CLAUDE.md (incremental update)

14.2 Session Storage (`utils/sessionStorage.ts`)

Message history is stored on disk in JSONL format, supporting chained recovery of compaction boundaries:

```
~/.claude/sessions/<session-id>/
├── messages.jsonl           ← Main message history
├── metadata.jsonl          ← Metadata (title, tags)
└── ...
```

Chained recovery: each message records a parentUuid pointing to its parent message. During recovery, starting from the newest message and following parentUuid backward forms a chain.

---

15. Persistent Memory Extraction

`extractMemories.ts` runs at the end of each complete query loop, writing key information to persistent storage.

15.1 Difference from Session Memory

	Session Memory	**Persistent Memory**
Location	`~/.claude/sessions/<id>/session-memory.md`	`~/.claude/projects/<path>/memory/`
Lifecycle	Current session	**Cross-session**
Content	Key work records of current session	Useful cross-session knowledge
Trigger	Context growth reaches threshold	End of each conversation turn (automatic)
Tool restrictions	Only Edit session memory file	Edit/Write in memory directory

"Each complete query loop" means: each time the main agent completes a conversation turn (model gives final response, no tool calls), stop hooks trigger extractMemories

15.2 Flow

```
extractMemories executes
    ↓
1. Check if main agent has already written memory file
   → Written → skip, advance cursor
   → Not written → continue
    ↓
2. Throttle check
   → Should not execute this round → return
```

```
     → Should execute → continue
      ↓
  3. Scan memory directory for existing files
      ↓
  4. Fork sub-agent
     → Read memory directory
     → Based on conversation delta (new messages after lastMemoryMessageUuid)
     → Generate update prompt
     → Use Edit/Write tools to write new memory files
      ↓
  5. Update cursor lastMemoryMessageUuid
```

15.3 Storage Structure and Content

```
~/.claude/projects/<path>/memory/
├── MEMORY.md              ← Index file (links to all topics)
├── auth-implementation.md ← Topic file 1
├── database-schema.md     ← Topic file 2
└── ...
```

Memory Types: Four Types

1. user — User Profile

```
  <name>user</name>
  - Content: user's role, goals, responsibilities, knowledge background
  - When to save: when learning user's role preferences, responsibilities, or knowledge
  - How to use: when answering questions, adjust style and depth based on user profile
  - Example:
    user: "I'm a data scientist investigating what logging we have in place"
    → Save: user is a data scientist, currently focused on observability/logging
```

2. feedback — Feedback Preferences

```
  <name>feedback</name>
  - Content: guidance on work approach given by user (what to avoid, what to keep doing)
  - When to save: when user corrects ("no not that") or confirms ("yes exactly")
  - Structure: rule itself → Why (reason) → How to apply (when applicable)
  - Example:
    user: "don't mock the database in these tests"
    → Save: integration tests must hit a real database, not mocks.
        Reason: prior incident where mock/prod divergence masked a broken migration
```

3. project — Project Status

```
  <name>project</name>
  - Content: ongoing projects, goals, bugs, incidents
  - When to save: when learning who is doing what, why, or by when
  - Structure: fact → Why → How to apply
  - Note: convert relative dates to absolute dates ("Thursday" → "2026-03-05")
  - Example:
```

```
    user: "we're freezing non-critical merges after Thursday — mobile team is cutting a
release branch"
    → Save: merge freeze begins 2026-03-05 for mobile release cut.
        Flag any non-critical PR work scheduled after that date
```

4. reference — External System References

```
  <name>reference</name>
  - Content: where information is located in external systems
  - When to save: when learning about external resources and their purpose
  - Example:
    user: "check the Linear project 'INGEST' if you want context on these tickets"
    → Save: pipeline bugs are tracked in Linear project 'INGEST'
```

Explicitly NOT saved content

```
  - Code patterns, conventions, architecture, file paths → can be derived from code
  - Git history, who-changed-what → git blame is authoritative
  - Debugging solutions → fix is in code; commit message has context
  - Content already written in CLAUDE.md
  - Ephemeral task details: in-progress work, temporary state
```

Meaning of Ephemeral tasks in the memory system:

- In-progress work
- Temporary state
- Current conversation context

Writing Format

Each memory write goes to its own file with frontmatter:

```
  ---
  name: Memory name
  description: One-line description
  type: user | feedback | project | reference
  ---

  Memory body content...

  MEMORY.md index records references:
  - [Memory Title](filename.md) — one-line hook
  - [Memory Title2](filename2.md) — one-line hook
```

15.4 Sub-Agent Execution

```
// Tool restrictions: createAutoMemCanUseTool(memoryDir)
allow: Read, Grep, Glob, read-only Bash
allow: Edit, Write within memoryDir
deny: all other operations
```

```
// Throttling control:
  // Execute once per N eligible turns
  // getFeatureValue('tengu_bramble_lintel', 1)
  // Default: execute every turn

// Max turns: 5 (prevent validation rabbit holes)
// skipTranscript: true (don't write to transcript, avoid races)
```

Note

The extractMemories sub-agent doesn't have a dedicated system prompt — it uses the parent agent's system prompt (coding assistant); task instructions are passed via user message:

```
You are now running as a memory extraction sub-agent.
Analyze the most recent ~{N} messages above and use them to update your persistent memory
system.

Available tools: Read, Grep, Glob, read-only Bash, and Edit/Write tools limited to paths
within the memory directory.

You must only use content from the most recent ~{N} messages. Don't waste turns on further
investigation or verification.

## Memory Types
[Detailed definitions of user / feedback / project / reference...]

## Content NOT to Save in Memory
[Code patterns / git history / content already in CLAUDE.md...]

## How to Save Memory
1. Read existing memory files first to avoid duplicates
2. Write to individual files in frontmatter format
3. Update MEMORY.md index (each entry max 150 chars)
```

15.5 Mutual Exclusion Protection

```
// If main agent wrote to memory file this round:
hasMemoryWritesSince(messages, lastMemoryMessageUuid)
// → Skip extraction, advance cursor
// → Both are mutually exclusive, avoiding conflicts
```

15.6 Wait on Exit

```
// Before CLI is exited by user, drainPendingExtraction()
// Wait up to 60 seconds for extraction to complete
// Prevent fork process from being killed during shutdown
```

---

16. History Management

`history.ts` manages the REPL's prompt history (for ↑ arrow recall and Ctrl+R search).

REPL: Read-Eval-Print Loop, i.e., the interactive terminal interface of Claude Code CLI

16.1 Key Constants

```
MAX_HISTORY_ITEMS = 100          // Max 100 items
MAX_PASTED_CONTENT_LENGTH = 1024 // Pasted content length limit
```

16.2 Main Functions

```
addToHistory(entry)                        // Add to history
getHistory()                               // Get current project history
getTimestampedHistory()                    // Timestamped version (for picker)
removeLastFromHistory()                    // Undo last addition
expandPastedTextRefs()                     // Resolve paste references
```

---

17. Message Normalization and Boundary Management

17.1 Message Normalization (`normalizeMessagesForAPI`)

This is the core function that converts internal message format to API request format:

```
normalizeMessagesForAPI(messages, tools): (UserMessage | AssistantMessage)[]
```

> **Note**
>
> Converts structured content in session to context for LLM

Execution Steps:

1. **Reorder attachments**: `reorderAttachmentsForAPI()` bubbles attachments to appropriate positions
2. **Filter virtual messages**:

   ```
   // The following types are filtered, not entering API requests:
     - progress messages
     - system messages (except local_command)
     - synthetic API error messages
   ```

3. **Handle local commands**: `local_command` system messages converted to user messages (merged into previous user message)
4. **Handle user messages**:
   - Merge consecutive user messages (Bedrock doesn't support consecutive user messages)
   - Strip tool_reference blocks (when tool search is disabled)
   - Strip blocks for failed attachments (PDF too long/password protected/invalid)
   - Handle text completions in tool_result (avoid stop sequence issues)
5. **Validate message order**: Ensure first message role='user'

17.2 Compaction Boundary Messages

```
// Create compaction boundary
createCompactBoundaryMessage(
  trigger: 'auto' | 'manual',
```

```
  preTokens: number,
  lastPreCompactMessageUuid?,
  userContext?,
  messagesSummarized?,
): SystemCompactBoundaryMessage

// Result structure:
{
  type: 'system',
  subtype: 'compact_boundary',
  content: 'Conversation compacted',
  compactMetadata: {
    trigger: 'auto' | 'manual',
    preTokens: number,
    userContext?: string,
    messagesSummarized?: number,
    preservedSegment?: {
      headUuid: UUID,
      anchorUuid: UUID,
      tailUuid: UUID,
    },
    preCompactDiscoveredTools?: string[],
  },
  logicalParentUuid?: UUID,  // Points to last message before compaction
}
```

Key fields:

- `logicalParentUuid`: establishes relationship between compaction boundary and last message before compaction, used for session storage chained recovery
- `preservedSegment`: tells loader how to restore preserved segment during deduplication skip
- `preCompactDiscoveredTools`: deferred tool schemas loaded before compaction

17.3 Microcompact Boundary Messages

```
createMicrocompactBoundaryMessage(
  trigger: 'auto',
  preTokens: number,
  tokensSaved: number,
  compactedToolIds: string[],
  clearedAttachmentUUIDs: string[],
): SystemMicrocompactBoundaryMessage
```

17.4 Finding Compaction Boundaries

```
// Find index of last compaction boundary message
findLastCompactBoundaryIndex(messages): number

// Get all messages after compaction boundary
getMessagesAfterCompactBoundary(messages, options?): T[]
// Includes the boundary message itself
// Supports HISTORY_SNIP: snip view also uses this slice
```

18. Key Design Decisions

18.1 Why Fork a Sub-Agent for Summary?

All background tasks (summary generation, memory extraction) are executed via `runForkedAgent`, core reasons:

1. **Prompt cache reuse**: fork inherits the main session's complete cache prefix (system prompt + tools + context messages), avoiding recalculation
2. **State isolation**: sub-agent cannot modify main agent's `readFileState`, skill list, or other internal state
3. **Interruptible**: user Esc interrupts sub-agent via shared `AbortController` signal
4. **No side effects**: sub-agent's messages don't enter main transcript (`skipTranscript: true`)

18.2 Why Microcompact at the API Layer Instead of Directly Modifying Messages?

Cached Microcompact chooses to operate at the API layer via `cache_edits` instead of directly modifying message content:

1. **Preserve prompt cache**: directly modifying message content would break server-side cache, causing recalculation every time
2. **Reduce network load**: API only needs to transmit delta changes (what to delete), not the entire prefix
3. **Atomicity**: server-side cache operations are atomic, avoiding race conditions
4. **Complementary to compaction**: microcompact cleans old results, full compaction handles accumulated pressure

18.3 Why Use GrowthBook for Dynamic Configuration?

All thresholds and switches use GrowthBook for dynamic configuration, not hardcoding:

```
getFeatureValue_CACHED_MAY_BE_STALE('tengu_session_memory', false)
getDynamicConfig_CACHED_MAY_BE_STALE('tengu_sm_compact_config', {})
```

- **Zero-delay deployment**: adjust thresholds without releasing new versions
- **A/B testing**: different user groups use different configurations
- **Gradual rollout**: progressively expand feature coverage
- **Non-blocking cache**: `CACHED_MAY_BE_STALE` returns cached value immediately, doesn't block initialization

> **Note**
>
> These parameter thresholds and switches are directly exposed interfaces; they can be adjusted directly without changing the underlying code

19. Global Data Flow Overview

19.1 Complete Request Processing Flow

Where is session memory compact??

```
User Input
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 1. Microcompact                                                      │
│    microcompactMessages(messages, toolUseContext, querySource)       │
│                                                                      │
│    ┌────────────────────┐    ┌──────────────────────────────────┐    │
│    │ Time-based MC      │    │ Cache Edit MC (CACHED_MICROCOMPACT) │  │
│    │ if (gap >= 60min) │    │ if (cache_edits API available)   │    │
│    │ Clear old tool results │ Delete old tools, keep cache prefix│  │
│    └────────────────────┘    └──────────────────────────────────┘    │
│                                                                      │
│    └─ Returns { messages (cleaned), compactionInfo: { pendingCacheEdits } } │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 2. Snip History Pruning (HISTORY_SNIP feature) -> Actually not enabled│
│    snipCompactIfNeeded(messages)                                     │
│    ├─ Calculate snip point (by tokens or message count)              │
│    ├─ Replace with SNIP_MARKER                                       │
│    └─ Return tokensFreed (passed to autocompact)                    │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 3. Token Counting & Threshold Detection                              │
│    tokenCountWithEstimation(messages) - snipTokensFreed              │
│    vs getAutoCompactThreshold(model)                                 │
│                                                                      │
│    ┌──────────────┐  ┌──────────────────┐  ┌────────────────────┐  │
│    │ < threshold  │  │ >= threshold &   │  │ >= blocking limit  │  │
│    │ Normal API call│ │ < blocking       │  │ Block input        │  │
│    │              │  │ Trigger warning/error│ │                  │  │
│    └──────────────┘  └──────────────────┘  └────────────────────┘  │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼ (exceeds threshold)
┌──────────────────────────────────────────────────────────────────────┐
│ 4. Session Memory Compact / Auto-Compact                             │
│    autoCompactIfNeeded(messages, toolUseContext, ...)                │
│                                                                      │
│    ┌────────────────┐    ┌──────────────────────────────────┐       │
│    │ SM-Compact?    │    │ Traditional Compaction           │       │
│    │ (use first)    │    │ compactConversation()            │       │
│    │ No API call    │    │ ├─ fork sub-agent to generate summary│   │
│    └────────────────┘    │ ├─ clear file read cache         │       │
│                          │ ├─ reinject files/skills/plan    │       │
│                          │ ├─ create compaction boundary    │       │
│                          │ └─ PostCompact hooks + cleanup   │       │
│                          └──────────────────────────────────┘       │
│                                                                      │
```

```
│    Success → reset circuit breaker consecutiveFailures = 0            │
│    Failure → consecutiveFailures++ → 3 times triggers circuit break  │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 5. API Call (queryModelWithStreaming)                                │
│    ├─ normalizeMessagesForAPI()                                       │
│    ├─ inject cache_reference + cache_edits (if any)                   │
│    ├─ send anthropic.beta.messages.create streaming request            │
│    └─ process streaming response                                     │
└──────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────┐
│ 6. Stop Hooks (background tasks, parallel execution)                  │
│                                                                      │
│    ┌──────────────────────┐  ┌──────────────────────────────────────┐│
│    │ extractMemories    │  │ extractSessionMemory (sessionMemory) ||
│    │ → Persistent memory │  │ → Update session memory Markdown   ||
│    │ → memory/ directory │  │ → ~/.claude/sessions/<id>/         ||
│    └──────────────────────┘  └──────────────────────────────────────┘│
│                                                                      │
│    Both are mutually exclusive: if main agent wrote to memory, skip extraction│
```

19.2 Compaction Message Construction Flow

```
Compaction Result CompactionResult
    │
    ├─ boundaryMarker        ← SystemMessage(type=compact_boundary)
    │       └─ compactMetadata: { trigger, preTokens, preservedSegment }
    │
    ├─ summaryMessages[]      ← UserMessage(isCompactSummary=true)
    │       └─ isVisibleInTranscriptOnly: true (not sent to API)
    │
    ├─ messagesToKeep[]       ← Most recent N messages (SM-Compact scenario)
    │
    ├─ attachments[]         ← AttachmentMessage[]
    │       ├─ postCompactFileAttachments (recently read files, max 5, 50K total)
    │       ├─ taskStatus (async agent status)
    │       ├─ plan_file_reference (plan file)
    │       ├─ plan_mode (plan mode reminder)
    │       ├─ invoked_skills (used skills, max 5K each)
    │       └─ deferredToolsDelta / agentListingDelta / mcpInstructionsDelta
    │
    └─ hookResults[]          ← HookResultMessage[]
            └─ results from processSessionStartHooks()
                └─ reload CLAUDE.md, memory files, etc.

Assembly: buildPostCompactMessages(result)
    [boundaryMarker, ...summaryMessages, ...messagesToKeep, ...attachments, ...hookResults]
```

19.3 Token Consumption by Context Layer

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Model Context Window                         │
│                                                                       │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ ┌──────────────┐    │
│  │ System prompt│ │ Tool definitions│ │ User context │ │ Message history│ │
│  │ ~30-50K      │ │ ~5-20K       │ │ ~5K      │ │ ~variable    │    │
│  └──────────────┘ └──────────────┘ └──────────┘ └──────────────┘    │
│                                                                       │
│  Message history composition:                                         │
│  ├─ Compaction boundary messages (fixed ~200B)                        │
│  ├─ Summary messages (structured text ~1-5K)                          │
│  ├─ Preserved tail messages (most recent N, 10K-40K)                  │
│  ├─ File attachments (max 5 files, 5K each)                           │
│  ├─ Skills attachments (max 25K total)                                │
│  ├─ Tool results after microcompact cleanup (only most recent few)    │
│  └─ Latest user message + assistant response                          │
└──────────────────────────────────────────────────────────────────────┘
```

Appendix A: Key File Index

File Path	Responsibility Description
`utils/tokens.ts`	Token counting (precise + estimation), API usage extraction
`utils/context.ts`	Context window constants and model-related calculations
`utils/contextAnalysis.ts`	Context statistical analysis, duplicate read detection
`services/tokenEstimation.ts`	Rough token estimation (including special type handling)
`services/compact/microCompact.ts`	Microcompact entry, time-triggered + cache edit
`services/compact/timeBasedMCConfig.ts`	Time-triggered MC GrowthBook config
`services/compact/apiMicrocompact.ts`	API layer context management strategies (clear_tool_uses, clear_thinking)
`services/compact/autoCompact.ts`	Auto-compact trigger, circuit breaker, threshold system
`services/compact/compact.ts`	Full compaction core logic (summary generation, file recovery)
`services/compact/prompt.ts`	Compaction summary prompt template
`services/compact/grouping.ts`	API round message grouping (by message.id)
`services/compact/postCompactCleanup.ts`	Post-compaction cache cleanup
`services/compact/compactWarningState.ts`	Compaction warning state management
`services/compact/compactWarningHook.ts`	React hook: subscribe to warning suppression state
`services/compact/sessionMemoryCompact.ts`	Session memory compaction (SM-Compact)
`services/SessionMemory/sessionMemory.ts`	Session memory extraction and update
`services/SessionMemory/sessionMemoryUtils.ts`	Session memory utilities, threshold calculation
`services/SessionMemory/prompts.ts`	Session memory template and prompt generation
`services/extractMemories/extractMemories.ts`	Persistent memory extraction

File Path	Responsibility Description
`context.ts`	System context (Git status), user context (CLAUDE.md)
`history.ts`	REPL prompt history management
`utils/messages.ts`	Message normalization, boundary messages, utilities
`utils/sessionStorage.ts`	Session message persistence
`services/contextCollapse/index.js`	Context Collapse (experimental, dynamically loaded)

Appendix B: All Configurable Constants

Constant	Value	File
`MODEL_CONTEXT_WINDOW_DEFAULT`	200,000	`utils/context.ts`
`COMPACT_MAX_OUTPUT_TOKENS`	20,000	`utils/context.ts`
`AUTOCOMPACT_BUFFER_TOKENS`	13,000	`services/compact/autoCompact.ts`
`WARNING_THRESHOLD_BUFFER_TOKENS`	20,000	`services/compact/autoCompact.ts`
`MANUAL_COMPACT_BUFFER_TOKENS`	3,000	`services/compact/autoCompact.ts`
`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES`	3	`services/compact/autoCompact.ts`
`POST_COMPACT_MAX_FILES_TO_RESTORE`	5	`services/compact/compact.ts`
`POST_COMPACT_TOKEN_BUDGET`	50,000	`services/compact/compact.ts`
`POST_COMPACT_MAX_TOKENS_PER_FILE`	5,000	`services/compact/compact.ts`
`POST_COMPACT_MAX_TOKENS_PER_SKILL`	5,000	`services/compact/compact.ts`
`POST_COMPACT_SKILLS_TOKEN_BUDGET`	25,000	`services/compact/compact.ts`
`MAX_PTL_RETRIES`	3	`services/compact/compact.ts`
`MAX_SECTION_LENGTH`	2,000	`services/SessionMemory/prompts.ts`
`MAX_TOTAL_SESSION_MEMORY_TOKENS`	12,000	`services/SessionMemory/prompts.ts`
`DEFAULT_SM_COMPACT_CONFIG.minTokens`	10,000	`services/compact/sessionMemoryCompact.ts`
`DEFAULT_SM_COMPACT_CONFIG.maxTokens`	40,000	`services/compact/sessionMemoryCompact.ts`
`MINIMUM_MESSAGETOKENS_TO_INIT`	10,000	`services/SessionMemory/sessionMemoryUtils.ts`
`MINIMUM_TOKENS_BETWEEN_UPDATE`	5,000	`services/SessionMemory/sessionMemoryUtils.ts`
`TOOL_CALLS_BETWEEN_UPDATES`	3	`services/SessionMemory/sessionMemoryUtils.ts`
`TIME_BASED_MC_CONFIG.gapThresholdMinutes`	60	`services/compact/timeBasedMCConfig.ts`
`DEFAULT_MAX_INPUT_TOKENS`	180,000	`services/compact/apiMicrocompact.ts`
`DEFAULT_TARGET_INPUT_TOKENS`	40,000	`services/compact/apiMicrocompact.ts`
`IMAGE_MAX_TOKEN_SIZE`	2,000	`services/compact/microCompact.ts`
`MAX_HISTORY_ITEMS`	100	`history.ts`
`MAX_STATUS_CHARS`	2,000	`context.ts`
`TOKEN_COUNT_THINKING_BUDGET`	1,024	`services/tokenEstimation.ts`